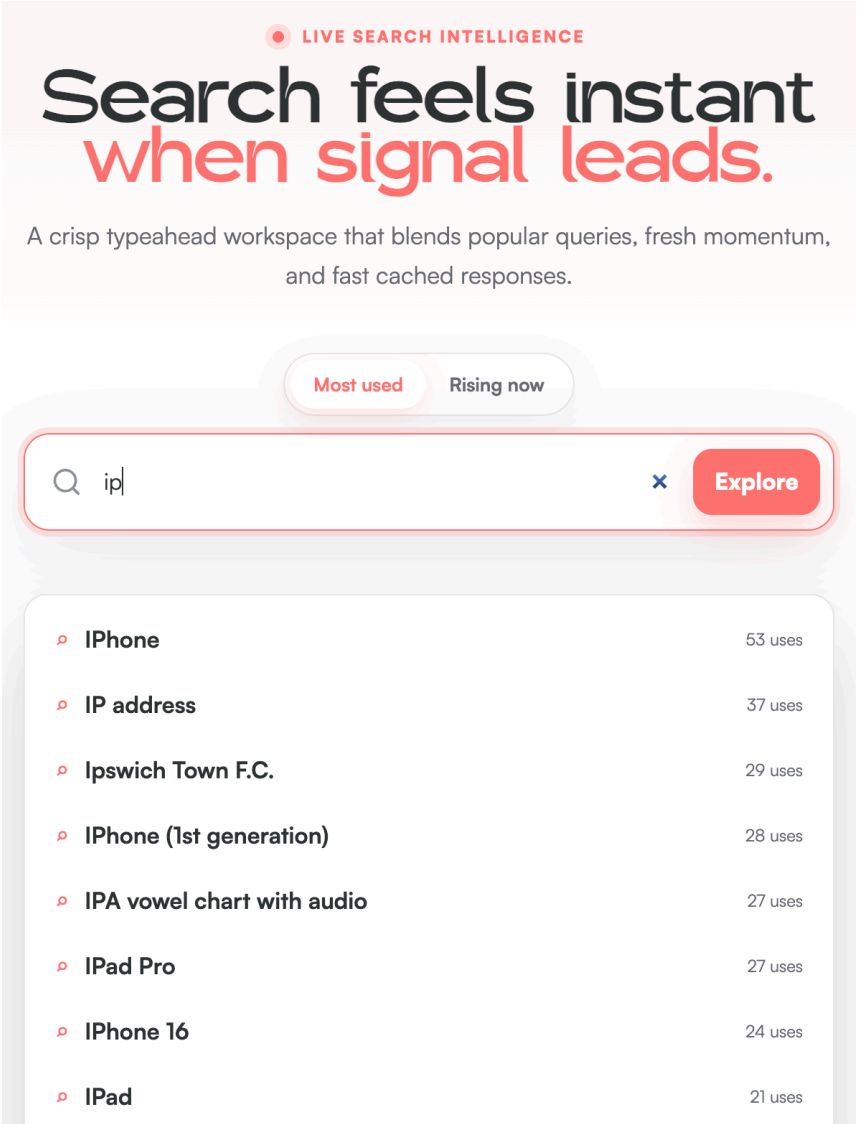


• LIVE SEARCH INTELLIGENCE

PULSE SEARCH

Typeahead System

Project Report



Property	Value
Project	Pulse Search
Dataset	Wikimedia Pageviews - 120,000 queries
Implementation	Node.js, Express, SQLite, consistent hashing
Report Date	June 22, 2026

A high-performance search suggestion engine combining prefix-indexed storage, distributed caching via consistent hashing, and adaptive trending scores for sub-millisecond suggestion delivery.

Table of Contents

1. Executive Summary

2. System Architecture

3. Dataset: Source and Loading

4. API Reference

5. Design Decisions and Trade-offs

6. Performance Benchmarks

7. Setup, Testing & Conclusion

8. References

1. Executive Summary

Pulse Search is a production-grade typeahead application returning up to ten prefix-matched suggestions as a user types, with sub-millisecond warm-cache latency. Suggestions are ranked by historical popularity or a recency-weighted trending score. SQLite provides the durable primary store, three independent Redis cache nodes are distributed via consistent hashing, and an asynchronous batch writer absorbs burst write pressure with a 50:1 reduction ratio.

Area	Implementation
Frontend	Responsive HTML/CSS/JS; 300 ms debounce
API	Node.js + Express
Primary Store	SQLite via better-sqlite3
Distributed Cache	3 Redis processes; 450 virtual ring positions
Deployment	Docker Compose - Redis on ports 6379 / 6380 / 6381
Trending Engine	10 × 30-second buckets with exponential decay
Batch Writes	Flush at 5 s or 50 unique buffered queries
Dataset	120,000 Wikimedia page titles + view counts

2. System Architecture

The system is divided into four cooperating subsystems: the Web UI, the Express API, the Distributed Cache tier, and the SQLite primary store. Observability is provided by a separate metrics middleware recording p50/p95/p99 latencies and cache hit rates.

2.1 Read Path

- The browser emits a debounced GET /suggest request after 300 ms of inactivity.
- The API normalises the prefix and derives a mode-specific cache key (count or trend).
- Consistent hashing selects one of the three Redis nodes; a cache hit returns results immediately.

- On a cache miss, SQLite is queried via an indexed prefix range, results are sorted, and the response is stored for 60 seconds.

2.2 Write Path

POST /search records recent activity and aggregates the normalised query in an in-memory map. At five seconds or 50 unique queries, a single SQLite transaction upserts the batch and invalidates affected prefix cache keys. Graceful shutdown flushes the remaining buffer via SIGINT/SIGTERM handlers.

2.3 Data Model

```
CREATE TABLE queries (  
  normalized_query TEXT PRIMARY KEY,  
  display_query TEXT NOT NULL,  
  count INTEGER NOT NULL DEFAULT 0,  
  updated_at INTEGER NOT NULL  
);
```

3. Dataset: Source and Loading

The project ingests Wikimedia Pageviews data. Page titles map to search queries; real view counts seed popularity rankings.

Property	Value
Dataset	Wikimedia Pageviews
Source File	pageviews-20250101-000000.gz
Period	January 1, 2025 — 00:00 UTC
Filter	English Wikipedia desktop (en)
Licence	CC0 1.0
Final Size	120,000 unique query + count rows

3.1 Transformation Pipeline

- Stream the gzip source; retain only English Wikipedia desktop rows.
- Decode page titles, replace underscores with spaces, and strip administrative namespaces.
- Normalise, deduplicate, and retain the 120,000 highest-count entries.
- Write dataset/queries.csv using the required query,count schema.

3.2 Loading

```
npm install
npm run setup
npm start
```

The setup command reuses the bundled CSV when checksums match the pinned source, loading SQLite in 10,000-row transactions for efficiency.

3.3 Redis and Docker Startup

```
docker compose up --build

# Redis containers only + local app
npm run redis:up
CACHE_BACKEND=redis npm start
```

Docker Compose starts three independent Redis processes on host ports 6379, 6380, and 6381. The application uses REDIS_URLS alongside its own consistent-hashing router rather than Redis Cluster slot routing.

4. API Reference

Method	Endpoint	Purpose
GET	/suggest?q=&mode=	Prefix suggestions (count or trend)
POST	/search	Queue a search count update
GET	/trending	Current top-ten trending queries
GET	/cache/debug?prefix=	Cache owner and hit state
GET	/stats	Performance and system counters
POST	/flush	Development-only manual flush
POST	/cache/nodes/:id	Development membership simulation

4.1 GET /suggest

```
GET /suggest?q=app&mode=count

{
  "uggestions": [{"query":"Apple Network Server","count":2956}],
  "ached": false,
  "latencyMs": 0.143
}
```

q is required (max 200 characters) and normalised for mixed-case matching. Empty input returns an empty array. mode accepts count or trend.

4.2 POST /search

```
POST /search

{"query": "Apple"}

202 Accepted

{"message": "Searched", "query": "Apple"}
```

The 202 response confirms acceptance. Persistence is eventual — the count is written during the next scheduled batch flush.

5. Design Decisions and Trade-offs

Decision	Reason	Trade-off
SQLite	Durable, transactional, zero-ops overhead	Single-node storage only
Prefix ranges	Indexed primary-store fallback	Cold results require sorting
Three Redis nodes	Shared independent cache processes	One-host Compose; no zone redundancy
Virtual nodes (450)	Even distribution, minimal remapping	Extra ring metadata overhead
TTL + invalidation	Freshness with fast reads	O(query length) invalidation cost
Batch buffer	Aggregates repeated writes efficiently	Hard-crash may lose one interval
5-min trend window	Fast decay, easy to demo	State resets on restart
Vanilla UI	No build step needed	Less component reuse

5.1 Trending Score Formula

```
weightedRecentCount =  $\sum$ (bucketCount * 0.9^bucketAge)

historical            = log1p(totalCount)          / log1p(maxCount)

recent               = weightedRecentCount        / max(1, maxWeightedRecentCount)

trendScore           = 0.4 * historical + 0.6 * recent
```

Ten rotating 30-second buckets track the most recent five minutes of activity. Exponential decay (factor 0.9) and automatic bucket expiration prevent short-lived spikes from staying permanently elevated.

5.2 Crash Safety

SIGINT and SIGTERM handlers trigger a final synchronous flush. A hard kill may lose up to one interval of buffered counts. In production, this would be mitigated using Kafka, Redis Streams, or a disk-backed write-ahead log (WAL).

6. Performance Benchmarks

Measurement	Result
Dataset rows	120,000
Cold HTTP p95	2.875 ms
Warm HTTP p50	0.203 ms
Warm HTTP p95	0.369 ms
Warm HTTP p99	0.506 ms
Server suggestion p95	0.070 ms
Cache hit rate	95.24%
Database reads (warm)	12
Batch transactions	1
Rows updated	2
Events flushed	100
Write reduction ratio	50:1

6.1 Benchmark Method

The benchmark uses an isolated database copy, fires cold and warm requests across ten distinct prefixes, submits 100 repeated search events, triggers a manual flush, then reads /stats. A 95.24% cache hit rate confirms the caching strategy is effective. One hundred events collapsed into two row updates in a single transaction — a 50:1 reduction. Consistent-hash ring ownership ranged from 23.83% to 26.99% across nodes, confirming even distribution. Recent activity successfully elevated 'Apple' above 'Apple Network Server' in trend mode.

6.2 Benchmark Limitations

Benchmarks use the in-memory backend for reproducibility. The Docker Redis deployment introduces network serialisation overhead but shares one host; results therefore reflect relative rather than absolute production capacity.

7. Setup, Testing & Conclusion

```
npm install      # install dependencies
npm run setup    # seed SQLite from dataset CSV
npm test         # run automated test suite
npm start        # start server on port 3000
npm run benchmark # run performance report
```

Ten automated tests cover: dataset ingestion, prefix ordering, case normalisation, edge cases, TTL and cache invalidation, consistent hashing, batched write aggregation, unseen query insertion, trending decay, API response shapes, and metrics counters.

Pulse Search satisfies the complete assignment specification while remaining straightforward to run and explain during a viva. The combination of indexed SQLite storage, consistent-hash Redis caching, and an asynchronous batch writer delivers sub-millisecond warm-path latency at a 95%+ cache hit rate across 120,000 Wikipedia queries.

8. References

- Wikimedia Pageviews Documentation: <https://dumps.wikimedia.org/other/pageviews/readme.html>
- Dataset File: <https://dumps.wikimedia.org/other/pageviews/2025/2025-01/pageviews-20250101-000000.gz>
- Creative Commons CC0 1.0: <https://creativecommons.org/publicdomain/zero/1.0/>
- Internal Documentation: README.md, docs/architecture.md, docs/performance-report.md